# Rapport du projet conteneurisation

Qinghui LIN Loïc HUANG Ines HARRAOUI

Sorbonne Université - Master RES - Avril 2026

## Contents

**1 Introduction** **2**

**2 Source des données** **2**
2.1 Notre choix : le dataset TMDB sur Kaggle . . . . . 2
2.2 Pourquoi ce choix ? . . . . . 2

**3 Architecture** **2**
3.1 Vue d'ensemble . . . . . 2
3.2 Arborescence du projet . . . . . 3
3.3 Choix du front-end : Vue + Vite . . . . . 3
3.4 Choix du back-end : FastAPI . . . . . 4
3.5 Choix de la base de données : PostgreSQL . . . . . 4

**4 Conteneurisation avec Docker** **4**
4.1 Utilisation du build multi-stage . . . . . 4

**5 Déploiement et CI/CD** **4**
5.1 Kubernetes . . . . . 4
5.2 Pipeline CI/CD . . . . . 5

**6 Schéma d'architecture** **5**

**7 Conclusion** **5**

# 1 Introduction

Ce projet consiste à construire une application web qui affiche un catalogue de films. L'utilisateur doit pouvoir parcourir le catalogue des films, effectuer des recherches et consulter les détails de chaque film. L'application doit également être découpée en trois parties : la base de données, le backend et le frontend, chacun fonctionnera dans son propre conteneur Docker.

# 2 Source des données

## 2.1 Notre choix : le dataset TMDB sur Kaggle

Pour notre application, nous devons choisir une source de données. Nous avions eu le choix entre deux options :

- **Option A - Utiliser une API (TMDB)** : Cela consiste à envoyer les requêtes à une API qui agit comme un intermédiaire entre l'utilisateur et la base de données. L'API récupère les données depuis la base de données pour ensuite le renvoyer à l'utilisateur.
- **Option B - Utiliser un Dataset existant** : Cela consiste à récupérer un ensemble de données déjà prêt à l'emploi.

Nous avons fait le choix d'utiliser un dataset (Option B). Le dataset utilisé est le *TMDB Movies Dataset 2024* trouvable sur Kaggle[1]. Cette ensemble de données regroupe plus d'un million de films et contient plusieurs informations dont nous avons besoin (titres, genres, notes, synopsis, etc...). Ce choix nous permet de travailler en local, sans dépendre d'un service externe, ce qui facilite grandement le développement et les tests.

## 2.2 Pourquoi ce choix ?

- **Rapidité et moins couteux en ressources** : Toutes les requêtes ne passent pas par un intermédiaire (API) pour pouvoir recupérer les données donc l'échange se fait directement entre l'utilisateur (backend) et la base de données. Les APIs sont souvent payantes ou gratuites mais avec un nombre limité de requêtes disponibles.
- **Contrôle total** : Nous définissons nous-mêmes la structure de la base de données.
- **Données riches** : Les nombreux champs disponibles permettent d'implanter des fonctionnalités avancées comme les filtres ou les recommandations.

# 3 Architecture

## 3.1 Vue d'ensemble

L'application est découpée en trois parties qui communiquent entre elles :

- **La base de données** (PostgreSQL) : Nous avions récupéré les données de l'ensemble de données Kaggle pour les mettre dans une table films.
- **Le backend** (FastAPI) : Reçoit les requêtes du navigateur, interroge la base de données et renvoie les résultats.
- **Le frontend** (Vue + Vite) : la partie visible dans le navigateur, qui affiche les films et envoie les recherches au serveur.

Chaque service tourne dans son propre conteneur Docker. Ils communiquent via un réseau interne Docker. Cela permet de redémarrer ou modifier un service.

[1] `https://www.kaggle.com/datasets/asaniczka/tmdb-movies-dataset-2023-930k-movies`

## 3.2 Arborescence du projet

La figure ci-dessous montre l'organisation des fichiers du projet. Le dossier app contient deux sous-dossiers principaux (backend et frontend). Le fichier docker-compose.yml contient la définition des conteneurs et leur configuration. Le dossier minikube_config contient toutes les configurations des pods, de l'ingress et de configmap.

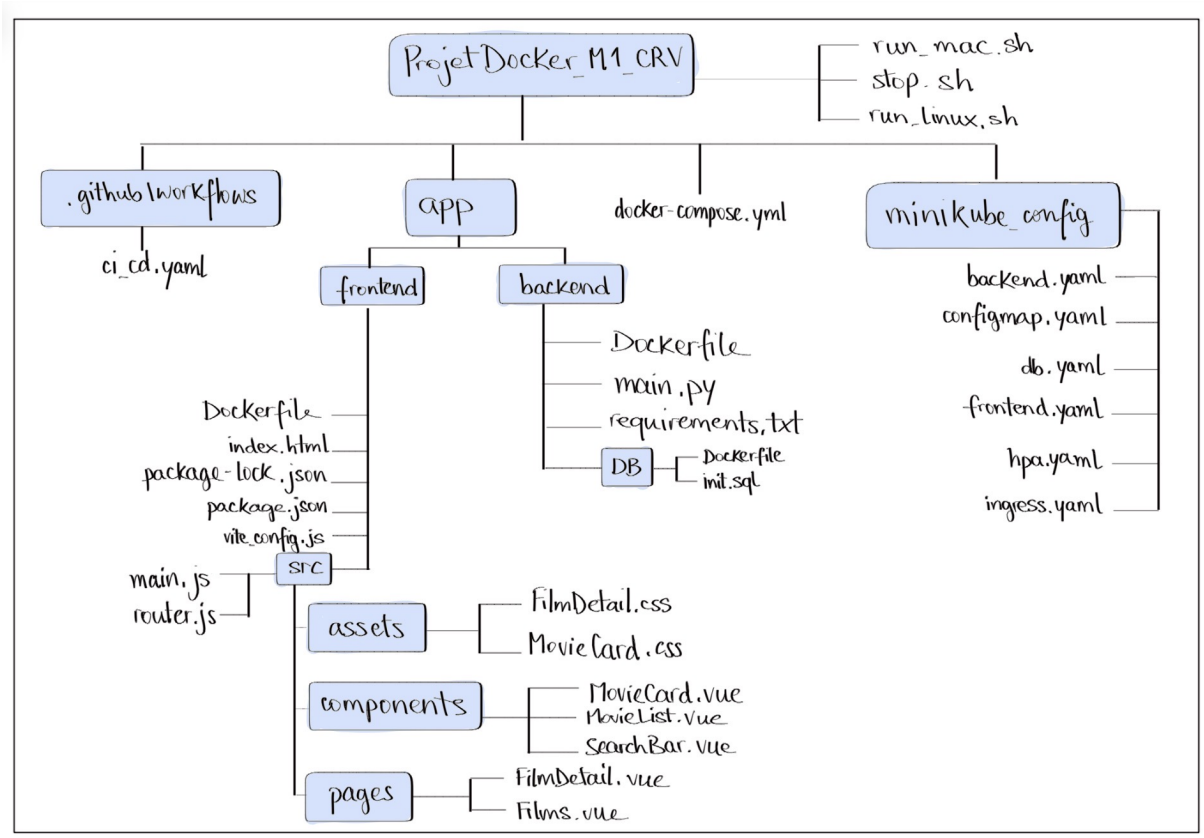



Figure 1: Arborescence des fichiers du projet

Le dossier backend regroupe le code du serveur (main.py) et la configuration de la base de données (init.sql). Le dossier frontend contient l'interface Vue, organisée en composants (MovieCard.vue, SearchBar.vue, ...), en pages (Film.vue, FilmDetail.vue), et en fichiers CSS pour gérer le style.

## 3.3 Choix du front-end : Vue + Vite

Le front-end est la partie que l'utilisateur voit dans son navigateur : le catalogue de films, la barre de recherche, les pages de détails. Nous avons choisi Vue + Vite pour les raisons suivantes :

- **Prise en main rapide** : Vue est facile à prendre en main et permet de pouvoir coder facilement l'interface qu'on veut avoir. Chaque composant est défini dans un fichier .vue qui regroupent HTML, CSS et JS.
- **Modifications visibles instantanément** : Toute modification du code est directement affichée dans le navigateur, sans avoir à recharger la page.

### 3.4 Choix du back-end : FastAPI

Le back-end est la partie qui traite les données et communique avec la base de données et l'interface. Quand l'utilisateur cherche un film, c'est le back-end qui reçoit la demande, interroge la base, et renvoie les résultats. Nous avons choisi FastAPI pour les raisons suivantes :

- **Gestion efficace des requêtes simultanées** : FastAPI peut traiter plusieurs demandes en même temps. Quand elle attend une réponse de la base de données, elle ne reste pas bloquée.
- **Vue en cours** : FastAPI a été vu en TME, ce qui nous a influencé sur le choix du backend. De plus, le fait que le code du back-end soit dans un seul fichier rend la lecture du code plus simple.

### 3.5 Choix de la base de données : PostgreSQL

La base de données stocke tous les films. Nous avons choisi PostgreSQL pour les raisons suivantes :

- **Structure adaptée** : Les données TMDB sont sous forme de tableau : Chaque ligne correspond à un film et chaque colonne donne ses caractéristiques, ce qui correspond parfaitement à une base de données relationnelle.
- **Import rapide** : PostgreSQL permet d'importer directement un fichier CSV grâce à la commande COPY, ce qui rend l'importation très rapide même s'il y a plusieurs millions de lignes.

## 4 Conteneurisation avec Docker

### 4.1 Utilisation du build multi-stage

Pour construire l'image Docker de la base de données, nous utilisons le build multi-stage qui se déroule en deux étapes :

1. Une première image, basée sur Python est utilisée pour télécharger le fichier de données depuis Kaggle.
2. Une seconde image, basée sur PostgreSQL, récupère le fichier de données et le script d'initialisation de la base de données, sans Python. PostgreSQL va exécuter automatiquement le script init.sql. Tout d'abord, il va importer les données du fichier CSV dans une table temporaire. Ensuite, il nettoie les données en remplaçant les champs vides par NULL et en convertissant les dates au bon format puis il insère les données dans la table finale en supprimant les doublons.

## 5 Déploiement et CI/CD

### 5.1 Kubernetes

En plus du lancement local avec Docker Compose, nous avons préparé un déploiement sur Minikube. Le dossier `minikube_config` contient un fichier YAML pour chaque composant de l'application :

- `db.yaml` : déploie la base de données PostgreSQL et l'expose sur le port 5432.
- `backend.yaml` : déploie le backend sur le port 8000.
- `frontend.yaml` : déploie l'interface sur le port 80.
- `configmap.yaml` : contient la configuration de Nginx. Les requêtes vers /api/ sont redirigées vers le back-end, le reste est envoyé vers le front-end.
- `ingress.yaml` : même rôle que configmap mais au nivau de Kubernetes. Les requêtes sont redirigées vers le bon service selon l'URL.
- `hpa.yaml` : adapte automatiquement le nombre de pods du back-end. Si l'utilisation du processeur dépasse 20 %, hpa va créer automatiquement de nouvelles instances, jusqu'à un maximum de 5.

### 5.2 Pipeline CI/CD

Le pipeline CI/CD est implémenté dans le fichier ci_cd.yaml. Il se déclenche à chaque push ou pull request pour vérifier :

1. **Build** : vérifie que les trois images Docker se construisent correctement.
2. **Minikube** : démarre un cluster Minikube, construit les images dans le cluster, crée un secret, applique tous les fichiers YAML et vérifie que les pods backend, frontend et db arrivent à l'état Ready.

## 6 Schéma d'architecture

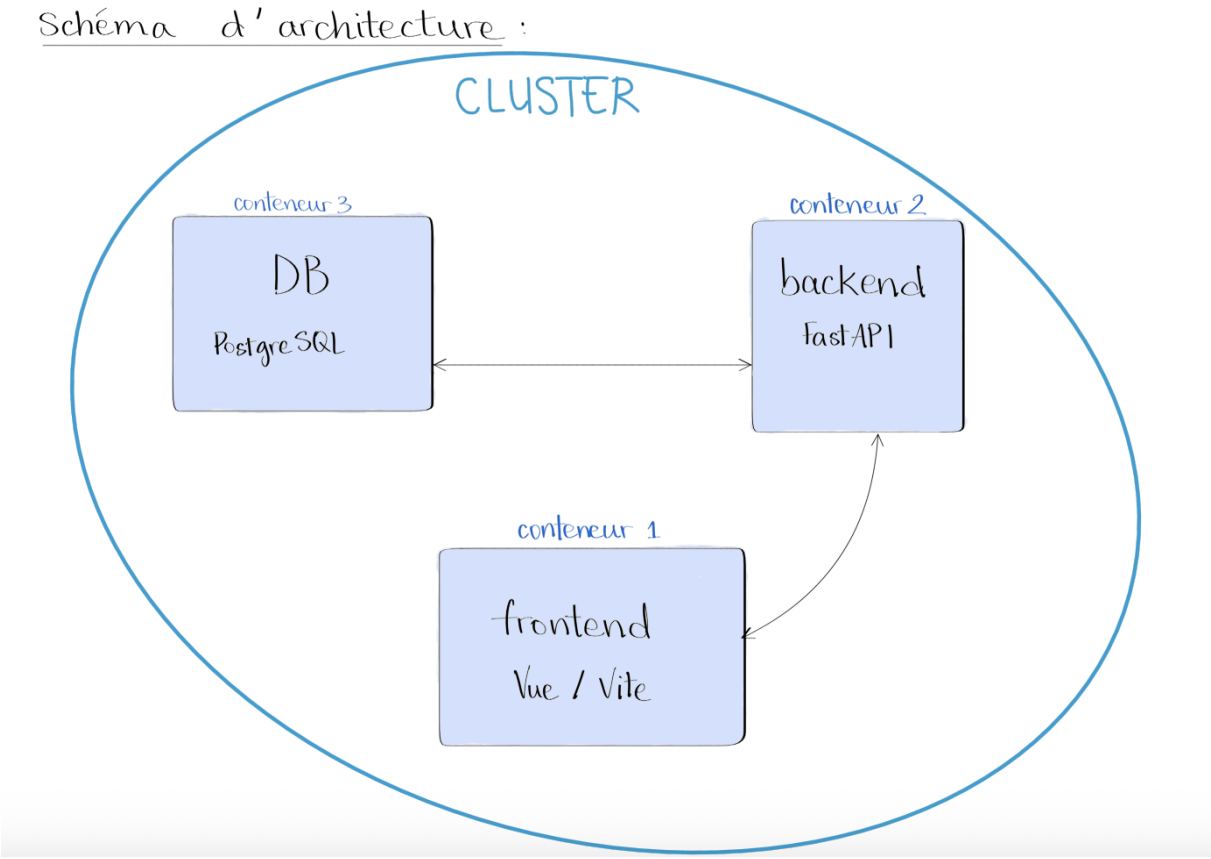



Figure 2: Architecture

## 7 Conclusion

Ce projet met en place une application web de catalogue de films conteneurisée et pouvant être lancée facilement avec une seule commande. Chaque choix technique a été fait pour privilégier la simplicité : un dataset local pour éviter la dépendance réseau, PostgreSQL pour sa rapidité d'import, FastAPI pour sa simplicité. La conteneurisation Docker permet à l'application de fonctionner de la même manière sur n'importe quelle machine, et Kubernetes sert a automatiser le deploiement et gérer les conteneurs.

Lien du github : `https://github.com/SniKoxx/ProjetDocker_M1_CRV`